ARGUS

Project Solution & Prototype Readiness Dossier

ARGUS — SIH26262 Team

October 2026

Contents

- Executive summary
- 1. Project charter
- 2. Solution specification
- 3. 24-hour scope and acceptance contract
- 4. Architecture and trust boundaries
- 5. Workflow state machine and rule catalogue
- 6. Data model
- 7. API contract outline
- 8. Security, privacy and regulatory boundaries
- 9. Prototype quality gate
- 10. Requirements traceability
- 11. Test plan
- 12. Deterministic demo test scenarios
- 13. Team workstreams
- 14. 24-hour build plan
- 15. Presentation source brief
- Appendix: Full-Stack Implementation Blueprint
  - 1. Current-state truth and target state
  - 2. Repository-first rule and default stack
  - 3. Required architecture
  - 4. Authentication and authorization requirements
  - 5. Shared database and schema requirements
  - 6. API and transaction contract
  - 7. Automation worker, reminders and notifications
  - 8. Evidence and document storage
  - 9. Production-shaped security baseline
  - 10. Deployment and configuration
  - 11. Test and acceptance plan
  - 12. Recommended implementation order for the 24-hour sprint
  - 13. Definition of done for coding agents
  - 14. Copy/paste execution prompt for a coding agent

Complete Visibility. Accountable Governance.
Proposed AICTE Approval & Compliance Intelligence Platform — not an official AICTE product.

Executive summary

New implementation addendum: The dossier PDF includes the full-stack implementation blueprint. The standalone versions are `PRODUCTION_IMPLEMENTATION_BLUEPRINT.md` and `ARGUS_Full_Stack_Implementation_Blueprint.pdf`. These define the backend/authentication/shared database implementation target; they do not mean those services are already running.

ARGUS proposes an automation-first platform for application routing, approval-stage visibility, EVC scheduling, time-bound undertakings, evidence review, document validity, deadlines, alerts/escalations and audit history. The primary technical demonstration is the event-to-action loop: event → validation → configured rule → duplicate prevention → permitted action → audit evidence → visible follow-up.

The 24-hour goal is not to deliver a production government system. It is to prove a focused vertical slice with fictional records, clear role boundaries, repeatable rule evaluation and honest simulation labels. Human officials retain statutory scrutiny and final decisions.

1. Project charter

1. Mission

Design an integrated, secure, configurable workflow and compliance platform that helps authorized teams track AICTE approval-related applications, scrutiny stages, EVC scheduling, institutional undertakings, evidence review, document validity, deadlines, alerts, escalations and audit history.

2. Problem statement fit

SIH26262 identifies fragmented coordination and follow-up across approval stages, stage-wise pendency, EVC work, time-bound undertakings, compliance evidence and prescribed document expiry. ARGUS responds by connecting case state, workflow rules, deadlines, task ownership, notifications and audit history in one workflow-oriented interface.

3. Primary users

- Institution user: view applicable cases and obligations, respond to a deficiency, submit evidence and renewal documents.
- Scrutiny officer: inspect case metadata, record observations, advance a permitted stage, or return a case with a reason.
- EVC coordinator/committee member: view eligible queue items, schedule a visit, track report submission and record completion.
- Regional/Bureau officer: monitor pendency and overdue matters, assign follow-up, review evidence and escalate under authorized policy.
- Competent authority: review the case file and record the statutory decision reserved to that role.
- Administrator/policy owner: configure authorized route profiles, demo thresholds, roles and escalation ladders; changes must be versioned and auditable in a production design.

Role labels in the browser prototype are simulated. Real authorization must be enforced server-side.

4. Product principle

Automate coordination, not statutory judgement. ARGUS can route a case when a configured prerequisite is fulfilled, create reminders, assign work and escalate according to approved rules. It must not autonomously grant/reject approval or substitute for scrutiny, inspection, verification, or a competent authority's decision.

5. Outcomes sought

- Clearer current status, owner, next action and due date per case.
- Fewer manually repeated follow-ups through deterministic reminders/escalations.

- Earlier visibility of EVC scheduling and report bottlenecks.
- Traceable compliance evidence submission and authorized verification.
- Advance notice of document expiry and an auditable renewal trail.
- More accountable case movements and policy-driven reporting.

These are intended outcomes, not measured results. No impact percentage should be claimed without evaluation evidence.

6. 24-hour prototype goal

Deliver a demo-ready vertical slice with synthetic cases where a user can: inspect a case; advance a permitted stage; return it for deficiency with a required reason; resubmit and resume; create an EVC scheduling task; submit and separately verify compliance evidence; trigger reminder and escalation rules; identify document-expiry risks; and inspect a correlated audit history.

7. Explicit non-goals for the 24-hour sprint

- Live AICTE/ERP or government-system integration.
- Production legal-rule engine or authority for statutory timing interpretation.
- Real email/SMS/WhatsApp delivery unless credentials/provider integration are truly configured and tested.
- Production auth, security certification, or multi-tenant production data isolation in a static mock.
- Definitive document-authenticity checking or autonomous AI approval/rejection.

8. Success definition

A jury member can follow one case from a clear event, through rule evaluation and a permitted action, to an audit record. The team can show positive and blocked/negative paths, repeat a cycle without duplicate alerts, explain role boundaries, and transparently distinguish what is implemented from what is simulated.

2. Solution specification

Executive summary

New implementation addendum: The dossier PDF includes the full-stack implementation blueprint. The standalone versions are `PRODUCTION_IMPLEMENTATION_BLUEPRINT.md` and `ARGUS_Full_Stack_Implementation_Blueprint.pdf`. These define the backend/authentication/shared database implementation target; they do not mean those services are already running. ARGUS is an automation-first approval workflow and institutional compliance console. It combines configurable application routes, case-stage state, EVC scheduling, time-bound undertaking tracking, document validity, alerts/escalations, role-aware work queues and audit records. The product’s core value is the closed loop between the event that occurs, the rule that matches, the permitted action that follows and the traceable evidence that the action occurred.

The operating loop

1. **Capture event:** stage completed, deficiency returned/resubmitted, EVC becomes eligible, evidence submitted, deadline nears/passes, document validity changes.
2. **Validate context:** target record exists; state transition is legal for the configured route; actor has permission; required fields exist; date source is clear.
3. **Evaluate a versioned rule:** inspect route profile, prerequisites, due/expiry dates, owner, configured threshold and escalation ladder.
4. **Prevent duplicates:** create an idempotency key such as `entity_id:rule_id:threshold_window:policy_version`; do not repeat an already processed action.
5. **Plan and authorize:** permitted routing/task/alert actions may run automatically; restricted scrutiny and final decision actions require an authorized human.
6. **Execute:** update case/task state, create in-app notification/task or queue an external notification (production only).

7. **Audit:** record actor/system identity, timestamp, target, event, rule/policy version, outcome, correlation ID, and reason where applicable.
8. **Surface exceptions:** failed actions, missing owner, invalid configuration and retry exhaustion must remain visible instead of disappearing silently.

Functional modules

A. Application intake and routing

- Capture synthetic/demo application metadata, institution, application type, region, owner, route profile, current stage, stage due date and status.
- Support route profiles representing applicable combinations, including `Scrutiny only`, `Scrutiny + EVC`, `EVC + SAC/SHC`, and `Only EVC`, plus an extensible configuration model.
- Compute next eligible stage from the selected profile rather than assuming all applications share one linear route.
- Block a transition if prerequisites or role permissions are missing.
- Return for deficiency only with a non-empty reason and record the stage to resume. Resubmission returns to the designated review point rather than silently skipping scrutiny.
- Never auto-complete a final statutory decision. Route to `Decision pending` and expose the decision action only to the competent-authority role.

B. Pendency and workload

- Display counts by stage/status, age band, region and owner.
- Expose current owner, next action, due date, days pending and whether reminder/escalation has fired.
- Provide case search/filter and an aging/bottleneck view. Metrics use seeded data in the prototype and are not performance claims.

C. EVC scheduling

- Put a case in the EVC queue only when the selected route requires EVC and configured prerequisites are complete.
- Track coordinator, scheduling state, visit date, panel/committee, report status and next action.
- Generate a scheduling task for eligible but unscheduled cases; remind/escalate if the configured timeline is crossed.
- Keep visit scheduling and report completion separate events.

D. Undertakings and compliance evidence

- Track obligation name/type, institution, applicable condition, due date, submission status, evidence reference, reviewer and verification state.
- Submission creates a pending-verification state; it must not mark evidence verified.
- Authorized reviewers record verified/rejected/needs-clarification with notes and timestamps.
- Missing, upcoming and overdue obligations participate in rule evaluation.

E. Document validity and renewal

- Track document type, institution, issue/expiry dates, renewal status and review owner.
- Produce advance expiry warnings and an overdue escalation according to configurable demo policy.
- Treat renewal submission and authorized verification as separate states.
- The prototype does not prove document authenticity; production authenticity checks would require authorized sources and explicit integration.

F. Alerts and escalations

- Supported demo categories: due-soon reminder, overdue escalation, EVC scheduling task, EVC report follow-up and document-expiry warning.
- Each alert must expose target, rule, reason/trigger, recipient role/owner, severity, status and audit reference.
- External delivery channels remain simulated unless provider delivery is implemented and verifiably tested.

G. Audit and reports

- Search/filter audit events by record, actor, event type, rule and timestamp.
- Export a CSV for demo review if supported by the application.
- A production event should be append-only, time-stamped, access-controlled and correlated to the triggering event and rule evaluation.

H. Rules administration

- Configure route profile, reminder threshold, expiry threshold, escalation recipient and stage SLA for the demo.
- Clearly mark example thresholds as `DEMO CONFIGURATION — NOT AN OFFICIAL AICTE TIMELINE`.
- In production, rule changes need authorization, versioning, effective dates, validation, audit and test/sandbox mode before activation.

Minimum end-to-end demo scenario

1. Open a synthetic case that has reached the scrutiny stage.
2. Complete the permitted stage as a scrutiny officer and show rule-driven transition to the next eligible stage.
3. Return another case for deficiency; omit the reason once to demonstrate validation, then provide a reason and confirm. Show the audit record and resubmission path.
4. Run the automation cycle on seeded deadlines. Show reminders and escalations, then run it again and prove duplicate actions are not created.
5. Show an EVC-required case in the queue, assign a visit and record report status.
6. Submit evidence as an institution user, then switch to a reviewer and verify it; show that submission did not equal verification.
7. Show an expiry warning, submit a renewal, and keep verification pending until an authorized action.
8. Open audit history and trace the events back to the case/rule.
9. Show a `Decision pending` case and explain that the competent authority—not an AI model—makes the final statutory decision.

Information architecture

- Overview: pendency, due soon, overdue, EVC queue, recent automation events.
- Applications: searchable list and case detail/action panel.
- Compliance: obligations and evidence verification states.
- EVC desk: eligibility, assignment, visit date and report follow-up.
- Documents: validity, renewal state and expiry warnings.
- Alerts: reminder/escalation queue with acknowledgement.
- Audit: chronological trace of user and system events.
- Rules: configurable demo settings and enabled rule catalogue.

Design qualities

Use professional high-contrast regulatory console styling, legible tables, consistent status chips, clear current/next stages, visible data freshness, empty/error/loading states, responsive behavior and accessible controls. Status must not rely on color alone. All demo data must be identified as synthetic. Avoid decorative AI claims that do not support a user task.

3. 24-hour scope and acceptance contract

Objective

Build the smallest credible vertical slice that proves the automation loop. A broad dashboard with inert buttons does not satisfy the MVP.

P0 acceptance criteria — all are required for a strong demo

ID	Acceptance criterion	Evidence to capture
P0-01	At least five fictional applications have route, stage, owner, due date and status.	Applications list and case detail
P0-02	Completing a permitted stage advances to the route-specific next stage, not a universal hard-coded path.	Before/after stage plus rule/event log
P0-03	Invalid transition or unauthorized role is blocked with an explanatory message.	Negative test
P0-04	Deficiency return requires reason, retains resume stage, and allows resubmission.	Return → response → resume trail
P0-05	Automation cycle creates an upcoming deadline reminder and an overdue escalation from seeded records.	Alert queue and audit events
P0-06	Re-running the same cycle does not duplicate action for the same rule/subject/window.	First cycle vs second cycle counts
P0-07	EVC-required cases are queued after prerequisites; scheduling and report status are separately tracked.	EVC desk
P0-08	Evidence submission creates pending verification; only an authorized reviewer can verify/reject.	Submit as institution role; review as official role
P0-09	Expiring/expired document creates warning/escalation; renewal submission does not silently equal verification.	Document register + audit
P0-10	Each action/alert exposes actor/system, target, timestamp, reason/rule, outcome and correlation/reference ID.	Audit trail
P0-11	Prototype clearly labels synthetic data, demo policy thresholds and simulated notifications.	Persistent disclaimer/README
P0-12	Core flow works from a fresh open/reload; errors, empty states and form validation are understandable.	Smoke-test checklist

P1 — do only after P0 works

- Persist demo state locally or through the chosen backend.
- Role-aware landing pages, filters for region/owner/stage/urgency and aging analysis.
- CSV export of audit data and a compact report.
- Rule editor with enabled/disabled rules and a visible policy version.
- Backend API, database migrations, actual auth enforcement and notification provider, only when properly integrated and tested.

P2 — explicitly future work unless genuinely implemented

Live AICTE integration; official regulatory-rule ingestion; production multi-tenant access control; external notification delivery; official document authenticity verification; AI/OCR; production scheduler/retry queue; deployment/security audit; user research and measured impact.

24-hour schedule

- **Hour 0–1:** inspect repository and baseline, agree types, owner roles, acceptance criteria and branch strategy.
- **Hours 1–4:** shared data model, app shell, seeded dataset and application list/detail.
- **Hours 4–8:** route profiles, valid/invalid transitions, deficiency return and resubmission.
- **Hours 8–12:** rule evaluator, due-date reminders, overdue escalations and idempotency.
- **Hours 12–15:** EVC queue, assignment/scheduling and report status.
- **Hours 15–18:** compliance submission/verification and document renewal/expiry handling.
- **Hours 18–20:** audit correlation, filters, role-gating checks and error/empty states.
- **Hours 20–22:** integration freeze, tests, bug fixes and clean demo dataset/reset.
- **Hours 22–23:** rehearsal with exact script and backup screenshots/video.
- **Hours 23–24:** final regression, known-limitations note, branch cleanup and tag/release.

If time slips, cut visual polish before cutting end-to-end logic or auditability. Cut AI/OCR, live messaging and integrations before P0. Keep the main branch demoable.

Demo readiness gate

Do not declare ready until two people independently perform the scripted flow, a second automation cycle creates no duplicate entries, the deficiency reason persists, evidence cannot be self-verified by the submitting role, all visible dates are consistent, and the team can explain the simulation boundaries in one sentence.

4. Architecture and trust boundaries

Prototype now

A single-page vanilla HTML/CSS/JavaScript demo with seeded synthetic data and browser-local state. This is appropriate for a fast, portable interaction demonstration. It is not a secure multi-user architecture and must not be deployed as a live regulatory system.

Recommended production shape

1. **Web client:** role-aware screens, form validation and evidence submission; client state is not authoritative.
2. **API layer:** authenticated application, workflow, EVC, compliance, document, alert and audit endpoints.
3. **Policy/workflow service:** versioned route profiles, transition guards, permission checks and policy evaluation.
4. **Automation worker:** scheduled due-date scans and event-triggered evaluations; durable queue, retries, dead-letter visibility and idempotency keys.
5. **Relational database:** transactionally store case state, submissions, assignments, rule versions, idempotency records and append-only audit events.
6. **Secure document storage:** least-privilege access, content type/size checks, malware scanning as feasible, retention policy and access auditing.
7. **Notification adapter:** in-app queue first; external channels only via approved provider/configuration with delivery status and retry policy.
8. **Observability:** structured logs, metrics, correlation IDs, health checks, failed job list and alerting.

Event-to-action path

`command/event` → schema validation → authentication/authorization → load current state/version → validate transition preconditions → evaluate configured rule version → calculate idempotency key → transactionally write state/task/audit → enqueue notification outbox item → worker delivers or records simulated result → monitoring surface exposes outcome.

For external messaging, use a transactional outbox or equivalent reliable delivery pattern. Never send a message and then lose the database record of its result.

Design principles

- Configuration-driven route profiles, not one global linear route.
- Explicit state machine, allowed transitions and guarded actions.
- Separate case workflow from post-approval obligations and document register.
- Human approval boundary is explicit and enforced in the API.
- Event version/correlation ID supports traceability.
- Idempotent command processing makes retries safe.
- Use soft deletion/archive only where retention policy permits; audit and regulatory retention needs must be confirmed.
- Keep rule configuration versions and effective dates.

Stack decision for the 24-hour sprint

First inspect the existing GitHub repository and preserve the stack and scripts already in use. If the repository is genuinely empty, use a simple, familiar stack supported by the team; document the chosen frontend, API and database before parallel implementation. Avoid introducing a distributed architecture or an external service just to look sophisticated. The core event/rule/action/audit loop matters more than stack branding.

Trust boundaries

- Browser state and role dropdowns are untrusted.
- Uploaded evidence is untrusted until checked and reviewed.
- Rule configuration is privileged input and must be validated/versioned.
- Notification provider responses can fail or be delayed.
- AI/OCR results are candidate observations only, never authoritative decisions.
- Any official-system integration requires authorization, documented API access, secure credentials and tested error handling.

5. Workflow state machine and rule catalogue

Route profiles (configurable examples)

The following are prototype route shapes to demonstrate configuration. Validate actual stage sequences and prerequisites against applicable official procedures before real use.

Profile	Demo stage sequence
Scrutiny only	Initial Scrutiny → Decision Pending
Scrutiny + EVC	Initial Scrutiny → EVC Scheduling → EVC Report Review → Decision Pending
EVC + SAC/SHC	EVC Scheduling → EVC Report Review → SAC/SHC Review → Decision Pending
Only EVC	EVC Scheduling → EVC Report Review → Decision Pending

`Institution response` is a controlled return state, not a universal route stage. A returned case stores `resume_stage`, the return reason, returning actor and timestamp. On resubmission, the case returns to its stored review stage and the new evidence enters the review queue. `Decision Pending` never auto-approves or auto-rejects.

Required guards

- Current stage must match the expected state/version.
- Selected route profile must contain the requested next stage.

- Prerequisites must be completed.
- Actor must be allowed for the command and target region/case.
- Deficiency reason must be non-empty and meaningful.
- Evidence submission cannot self-verify.
- EVC task cannot be scheduled before EVC eligibility.
- Duplicate command/event cannot create a second side effect.
- Decision action is restricted to the designated competent-authority permission.

Rule catalogue

Rule ID	Trigger/condition	Permitted action	Guard/evidence
WF-01	A stage completion event arrives and route has next stage	Move to next eligible stage or create next-stage task	Validate actor, preconditions and route; write transition audit
WF-02	Authorized user returns for deficiency	Set Institution Response state and preserve resume stage	Mandatory reason; notify institution (simulated in prototype); audit
WF-03	Institution submits requested response	Restore resume stage and assign reviewer	Record submission reference and actor; not auto-verify response
DL-01	Unresolved item within configured reminder window	Create reminder/task	Threshold is demo config; idempotent per item/rule/ window/version
DL-02	Unresolved item past due and not already escalated at that level	Create escalation to configured recipient	Record due date, overdue duration and escalation rung
EVC-01	Route requires EVC and prerequisites are satisfied	Add/update EVC scheduling queue task	Do not create duplicate active queue items
EVC-02	EVC visit date near/past or report outstanding	Create reminder/escalation	Track scheduled date and report status separately
CMP-01	Institution submits evidence	Set Pending Verification and assign authorized reviewer	Submission must not equal verification
CMP-02	Authorized reviewer verifies/ rejects evidence	Set result with note and timestamp	Permission + audit record required
DOC-01	Document expiry within configured threshold	Create expiry reminder and renewal task	Rule config is illustrative unless validated
DOC-02	Document expired and renewal not verified	Create overdue alert/escalation	Idempotent; no claim authenticity is checked
SEC-01	Actor attempts a forbidden action	Deny action and record security-relevant audit event where policy permits	Server-side authorization required in production
SYS-01	Any automated action executes	Write correlated audit outcome	Unique idempotency key, rule version, result

Idempotency

Construct a unique action key, for example `subjectType:subjectId:ruleId:thresholdWindow:policyVersion`. Store it under a unique constraint before side effects are confirmed. Reprocessing the same event must return the earlier outcome, not send another notification or create a duplicate task. If business rules require a later escalation rung, include the rung/ window in the key.

Rule configuration

Each rule needs: `rule_id`, `version`, `enabled`, `effective_from`, optional `effective_to`, trigger, condition expression or typed fields, action, target role/queue, threshold, working-day/calendar policy, idempotency template, severity, and audit metadata. In production, use typed validated configuration rather than executing arbitrary code supplied by an administrator.

Demo date policy

Seeded dates and default thresholds exist only to make the demo repeatable. Label them `DEMO CONFIGURATION — NOT AN OFFICIAL AICTE TIMELINE`. Keep all dates in ISO format in data; display them in the user's locale. Tests must cover before/at/after threshold and timezone boundaries.

6. Data model

This is a logical model; use the existing repository's ORM and conventions where applicable. IDs are stable identifiers, timestamps are UTC instants in production, and all policy/rule references must be versioned.

Core entities

ApplicationCase

- `id`, `institution_id`, `application_type`, `region_id`, `route_profile_id`, `route_profile_version`- `current_stage`, `status`, `assigned_role`, `assigned_user_id` (nullable), `created_at`, `updated_at`- `current_stage_started_at`, `due_at`, `version` (optimistic concurrency)- `return_reason` (nullable), `resume_stage` (nullable), `returned_by`, `returned_at`, `resubmitted_at`- Do not use institution names alone as IDs; avoid real personal data in demo fixtures.

RouteProfile / RouteStage

- `id`, `name`, `version`, `enabled`, `effective_from`, `effective_to`- ordered stage nodes with prerequisites, eligible roles, transition guards and next-stage conditions- policy source/reference and approval/change metadata

WorkflowEvent

- `id`, `event_type`, `subject_type`, `subject_id`, `actor_type`, `actor_id`- `occurred_at`, `correlation_id`, `payload_schema_version`, safe structured payload

RuleDefinition / RuleEvaluation

- `rule_id`, `version`, `enabled`, `configuration`, `effective_from`, `effective_to`- evaluation result: matched/not matched, condition summary, rule version, target, timestamp and correlation ID- avoid persisting unnecessary personal data in payload snapshots

Action / Notification

- `id`, `subject`, `action_type`, `recipient_role`, `recipient_id`, `status`, `severity`, `reason`- `rule_id`, `rule_version`, `idempotency_key`, `created_at`, `executed_at`, `outcome`, `failure_code`, `retry_count`- external delivery status is distinct from a generated in-app action/task

ComplianceObligation / EvidenceSubmission

- obligation: `id`, `institution_id`, `type`, `condition_reference`, `due_at`, `status`, `owner_role`, `reviewer_id`- submission: `id`, `obligation_id`, `submitted_by`, `submitted_at`, `evidence_ref`, `notes`, `status`- verification: `verified_by`, `verified_at`, `decision`, `review_note`- submission and verification are separate entities/states

DocumentRecord / RenewalSubmission

- document: `id`, `institution_id`, `document_type`, `issued_at`, `expires_at`, `status`, `owner_role`- renewal: `id`, `document_id`, `submitted_at`, `evidence_ref`, `review_status`, `reviewer_id`, `reviewed_at`, `review_note`- a metadata expiry alert is not proof of document authenticity or validity beyond the recorded dates

EVCRecord

- `id`, `application_id`, `eligibility_status`, `queue_entered_at`, `coordinator_id`, `committee_id`, `scheduled_at`, `visit_status`, `report_status`, `report_submitted_at`, `follow_up_due_at`

AuditEvent

- `id`, `event_id`, `actor_type`, `actor_id`, `action`, `subject_type`, `subject_id- occurred_at`, `correlation_id`, `rule_id`, `rule_version`, `reason`, `result`, minimal metadata- production design should make audit records append-only for application identities and restrict access based on policy

User / RoleAssignment / RegionScope

- authentication subject, active status, role(s), institution/region scope, permissions, assignment history- use least privilege, server-side checks and revocation; never trust the client role selector

Important relationships

- Institution 1:N ApplicationCase, ComplianceObligation, DocumentRecord.
- ApplicationCase 1:N WorkflowEvent, EVCRecord, AuditEvent.
- ComplianceObligation 1:N EvidenceSubmission.
- DocumentRecord 1:N RenewalSubmission.
- RuleDefinition 1:N RuleEvaluation and Action.
- Each Action and audit event should correlate to the triggering event where available.

Data quality and retention

Use explicit enums and foreign keys; validate due/expiry dates; retain raw time and source reference for date changes; record who changed a date and why; avoid hard deletes of records that may require audit retention; apply the retention policy only after it is authorized. Encrypt sensitive data in transit and at rest in a production deployment.

7. API contract outline

The contract is framework-neutral. Use the naming/serialization conventions of the repository's current backend if already established. Every mutating route needs authentication, permission checks, input validation, transaction boundaries and audit handling.

Read endpoints

- `GET /api/v1/dashboard/summary?regionId=...` — stage counts, overdue/due-soon counts, EVC queue size; return data timestamp and applied scope.
- `GET /api/v1/applications` — filters: stage, status, region, owner, query, due bucket; return cursor/page and total where practical.
- `GET /api/v1/applications/{id}` — case plus route, next permitted actions, obligations/EVC link and recent audit references.
- `GET /api/v1/evc-queue` — only EVC-eligible cases; filter by scheduling/report status.
- `GET /api/v1/compliance-obligations` — filter institution/status/due bucket/reviewer.
- `GET /api/v1/documents` — filter expiry bucket/renewal state.
- `GET /api/v1/alerts` and `GET /api/v1/audit-events` — authorized filters, pagination and safe projections.
- `GET /api/v1/rules` — active version and authorized rule metadata.

Commands / mutations

- `POST /api/v1/applications` — create intake record; server calculates route starting state.
- `POST /api/v1/applications/{id}/complete-stage` — body includes expected version, completion summary and optional evidence refs; server validates route and advances to next configured stage.

- `POST /api/v1/applications/{id}/return-for-deficiency` — requires `reason`, expected version and optionally specific missing items; stores resume stage.
- `POST /api/v1/applications/{id}/resubmit` — references the deficiency request and evidence refs; resumes the designated review stage.
- `POST /api/v1/evc-queue/{id}/schedule` — committee/panel, date/time and coordinator; only eligible records.
- `POST /api/v1/evc/{id}/report` — report metadata/evidence reference and submission event.
- `POST /api/v1/compliance-obligations/{id}/evidence-submissions` — institution submission, creates Pending Verification.
- `POST /api/v1/evidence-submissions/{id}/verification` — authorized reviewer decision, note and outcome.
- `POST /api/v1/documents/{id}/renewal-submissions` — submit renewal evidence; status stays pending review.
- `POST /api/v1/renewal-submissions/{id}/verification` — authorized reviewer outcome.
- `POST /api/v1/automation/run` — admin/demo-only trigger; production cycles should normally be scheduled by a worker and run with service identity.
- `POST /api/v1/alerts/{id}/acknowledge` — acknowledgement is not the same as resolving the underlying issue.
- `POST /api/v1/rules/{id}/versions` — privileged change, validation, versioning and audit.

Common response envelope

Success: `{ "data": ..., "meta": { "requestId": "...", "asOf": "ISO-8601" } }`.

Error: `{ "error": { "code": "INVALID_TRANSITION", "message": "...", "details": { ... } }, "meta": { "requestId": "..." } }`.

Use stable codes such as `VALIDATION_ERROR`, `FORBIDDEN`, `NOT_FOUND`, `CONFLICT_VERSION`, `INVALID_TRANSITION`, `PREREQUISITE_NOT_MET`, `DUPLICATE_ACTION`, `POLICY_INVALID` and `TEMPORARY_UNAVAILABLE`.

Invariants

- `expectedVersion` prevents two concurrent users from silently overwriting stage state.
- Every accepted command returns a correlation/request ID.
- State-changing endpoints emit an audit event in the same transaction or via a reliable outbox.
- Notification sending must not occur before authorization and durable action/audit registration.
- Do not leak evidence/document content in list responses; use authorized time-limited access where storage is involved.
- Avoid status-changing `GET` requests.
- A route response must include allowed next actions and explanation, but server checks the rule again when command is submitted.

8. Security, privacy and regulatory boundaries

Prototype truth

The single-file browser app is not secure for real institutional data. Its role selector, localStorage state, sample users and client-side buttons can be changed by the browser user. The prototype is only for synthetic demo records.

Production requirements

- Authenticate users with an approved identity provider; use server-side authorization for every request.
- Apply least privilege and institution/region/case scoping. Enforce authorization on object IDs to prevent IDOR-style access.
- Separate role capabilities: institutions submit/observe their records; scrutiny roles perform allowed scrutiny actions; EVC roles manage eligible visit tasks; reviewers verify evidence; competent authority makes reserved decisions; admins manage policy only within authority.

- Store secrets in environment/secret management, never source control. Rotate and revoke exposed secrets.
- Validate inputs and route transitions server-side; use CSRF-safe sessions or proper token flows, output encoding, rate limits and dependency updates.
- Upload controls: allowlisted file types, size limits, MIME/content verification, malware scanning where feasible, encrypted storage, scoped downloads and access logs.
- Encrypt traffic and sensitive stored data; minimize personal information and redact logs.
- Log authentication, access-denial, data export, rule changes, evidence review and workflow transitions; avoid logging full sensitive document contents.
- Audit records should be append-only to application identities, query-controlled and retained according to the applicable approved policy.
- Protect notification endpoints; prevent injection, recipient spoofing and excessive messages. Keep delivery outcomes visible.
- Use backup/restore, incident response, access review, monitoring, rate limiting and vulnerability management before production.

Human oversight

- No model should autonomously grant/reject an application or substitute for statutory scrutiny, inspection or competent authority decisions.
- AI/OCR findings must be advisory, explain uncertainty, link to the source evidence/page/region and support human confirmation/correction.
- Do not infer legal non-compliance from OCR alone. Missing or conflicting fields create review tasks, not automatic adverse decisions.

Regulatory configuration

Use only verified, currently applicable official rules and circulars. Store source citation, rule version, effective date, owner and approval history with policy. A demo rule threshold is not a legal claim. Any official-system integration must be authorized and documented; do not scrape or impersonate official interfaces.

Privacy by design

Use synthetic records for hackathon judging. Avoid real names, personal identifiers, institution confidential documents, credentials, private emails and phone numbers. Define purpose, access scope, retention and deletion before handling real data.

9. Prototype quality gate

This is an internal acceptance rubric, not a promise of jury scoring. Each point is earned only when a test or reproducible demo proves it.

Point	Gate	Evidence needed
1	Problem fit and scoped product story	Team maps every core screen/action to SIH26262 and avoids invented impact claims
2	Route-aware workflow	Valid stage completion follows the selected route; invalid/missing prerequisites are blocked
3	Deficiency loop	Blank reason blocked; reason, resume stage, response submission and return to review are preserved

Point	Gate	Evidence needed
4	Automation correctness	Reminder/overdue/expiry rules evaluate exactly at/before/after thresholds using an explicit simulation clock
5	Idempotency	Re-running unchanged event inputs creates no duplicate action for same subject/rule/threshold window/policy version
6	EVC lifecycle	Eligibility, scheduling, visit date and report status are distinct; future visit cannot be recorded as already reported
7	Compliance lifecycle	Submission, pending review, verified/ rejected/clarification and resubmission are distinct and auditable
8	Document lifecycle	Expiry warnings, renewal submission, authorized review and renewal resubmission operate as separate states
9	Access and auditability	Role controls block prohibited actions in UI; every state change has actor/time/ target/rule/outcome/reference; production security limits explained
10	Reliability and demo readiness	Fresh/reset data works, all P0 scenario checks pass twice, responsive UI is usable, no console errors in the main scenario, limitations stated accurately

Pass rule

Target 10/10 requires a checkable evidence item for every row. If a criterion is implemented only in documentation, marked future, or not tested, it does not earn a point. This prevents presentation polish from being mistaken for software completeness.

Required evidence packet

- Commit SHA and test environment.
- P0 test checklist with pass/fail and reproducible notes.
- Screenshots or recording of route transition, deficiency return/resubmit, first and second automation cycle, EVC schedule/report, evidence verification, document renewal review and audit trace.
- Browser console output or automated test report.
- Limitations statement confirming local data, no production auth, no live AICTE integration and no external notification delivery unless those capabilities were separately built and verified.

10. Requirements traceability

Status meanings: **Prototype** = locally demonstrated behavior; **Planned** = specified but not yet implemented in production code; **Out of scope** = explicitly not claimed.

SIH26262 need	ARGUS requirement	Primary implementation area	Evidence to show	Boundary
End-to-end application routing	Route-specific state machine and guarded completion	`prototype/`; `technical/WORKFLOW_AND_RULES.md`	Complete stage, see next route-specific stage and audit record	Route examples need official validation
Deficiency return/revert	Reason required; retain resume stage; resubmit to review	Case detail and workflow commands	Blank reason blocked; full round-trip trail	No legal conclusion generated
Stage-wise pendency	Due-date status, owner, stage and aging overview	Overview, applications	Filter and sorted due/overdue cases	Demo metrics use synthetic records
EVC queue and reminders	Eligibility gate, schedule fields, report status	EVC desk, `EVC-01/02`	Eligible case queued, scheduled, report recorded	External calendar/notification not connected
Time-bound undertakings	Obligation, due date, evidence submit/verify states	Compliance module, `CMP-01/02`	Submit leaves Pending Verification; reviewer closes	Evidence authenticity not determined
Lease/document expiry	Expiry warning, renewal submit/verify, overdue escalation	Documents, `DOC-01/02`	Expired/upcoming warning and review trail	Recorded dates only; no external validation
Notifications and escalations	Configurable demo thresholds, generated in-app queue	Rules, alerts, rule cycle	Run cycle twice; actions and duplicates suppressed	No real email/SMS sent
Role-based dashboards/actions	UI role simulation and action gating	Role selector and action controls	Show allowed/blocked controls per role	Production authorization must be server-side
Audit trail	Actor, timestamp, target, rule, outcome, correlation	Audit panel and generated events	Trace same ID from event/alert to audit	Production append-only DB not yet present
Integration with existing authorized systems	Documented production adapter boundary	`technical/ARCHITECTURE.md`, `API_CONTRACT.md`	Show future integration plan, not a false claim	Out of scope for browser prototype
Secure storage and confidentiality	Synthetic data; production security spec	`technical/SECURITY_PRIVACY.md`	Point out trust boundaries	Prototype is not safe for live data
Human regulatory oversight	Decision Pending stage; decision action reserved to competent role	Case detail	Show no automatic final decision	Final statutory decision remains human

Release evidence checklist

- [ ] Run each P0 scenario in `technical/DEMO_TEST_SCENARIOS.md`.
- [ ] Record browser/version, date and commit SHA.
- [ ] Capture screenshot/video of route transition, deficiency round trip, repeated automation cycle, EVC, evidence verification, expiry and audit.
- [ ] Note every unimplemented feature as planned, not completed.
- [ ] Confirm every rule threshold displayed in the UI is labelled demo-only.

11. Test plan

Test levels

- Unit: transition guards, route selection, deadline threshold calculation, idempotency-key creation, permission checks and audit mapping.
- Integration: commands update state and audit together; rule evaluation creates action/outbox entries; duplicate events are safe; evidence verification cannot bypass role checks.
- UI/component: field validation, empty/error/loading states, role-aware controls, filters and accessible labels.
- End-to-end: full representative application journey, compliance/document lifecycle, EVC scheduling and repeat automation cycle.
- Security: forbidden object access, unauthorized transitions, role privilege escalation, upload misuse and secret leakage.
- Regression: route profile changes, timezone/daylight boundary, due dates at 0/1 threshold days and escalation ladder windows.

Required negative tests

1. Attempt stage completion when prerequisites are missing → blocked, state unchanged, explain reason.
2. Return for deficiency with blank reason → blocked.
3. Resubmit a case without an existing deficiency request → blocked.
4. Institution user attempts evidence verification → forbidden.
5. Case with route not requiring EVC attempts schedule → blocked.
6. Run the same automation event twice → one action per idempotency key.
7. Run the cycle at threshold boundary → correct inclusive/exclusive behavior specified by rule.
8. A notification provider failure → persisted action and failed/retry state remain visible.
9. Change a rule version mid-cycle → each action references the exact evaluated version.
10. Attempt final decision from a non-competent role → forbidden.

Manual smoke checklist

- [ ] Fresh open shows demo disclaimer and seeded data.
- [ ] Search/filter applications works and case details match list state.
- [ ] Valid stage completion changes only the allowed stage and creates audit event.
- [ ] Deficiency reason validation, return, resubmission and resume are correct.
- [ ] EVC eligibility, scheduling and report status are distinct.
- [ ] Evidence submission is pending verification until an authorized review action.
- [ ] Renewal submission remains unverified until a reviewer action.
- [ ] First automation cycle generates seeded matches; second adds no duplicates.
- [ ] Every action has a visible source rule and audit reference.
- [ ] Reset restores a known demo state.

- [ ] Mobile width has usable navigation/tables/buttons.
- [ ] Demo labels avoid false claims of live integrations.

Test evidence record

Record commit SHA, environment/browser, test date, commands, pass/fail count, known failures and screenshots/video. Don't claim a test passed without running it.

12. Deterministic demo test scenarios

Use only the seeded fictional dataset. The browser prototype includes a reset action so the cases can be replayed consistently.

Scenario	Actor	Action	Expected result
Happy-path route	Scrutiny Officer	Complete Initial Scrutiny for an eligible case	Moves to next route-specific stage; workflow rule/event and audit entry visible
Wrong actor	Institution User	Attempt officer-only stage completion	Denied in demo UI with clear explanation; no state change
Deficiency validation	Scrutiny Officer	Return a case without reason	Form blocks action
Deficiency round trip	Scrutiny Officer → Institution User → Scrutiny Officer	Return with reason, submit response, resume review	Reason and resume point preserved; each transition audited
EVC eligibility	Committee Coordinator	Open EVC desk	Only EVC-route cases in eligible stage are actionable
Schedule EVC	Committee Coordinator	Assign date and panel to an unscheduled case	Queue state updates; audit event appears
Evidence separation	Institution User → Regional Officer	Submit then verify evidence	Submission status becomes pending; only reviewer can verify; two events recorded
Renewal separation	Institution User → Regional Officer	Submit renewal	Expired/expiring document remains pending review until verified
Upcoming reminder	System/demo operator	Run cycle with due date inside threshold	Reminder action appears once with source rule and due-date reason
Overdue escalation	System/demo operator	Run cycle with unresolved past-due obligation	Escalation appears with recipient role and overdue amount
Idempotent repeat	System/demo operator	Run same cycle again without changing input	No duplicate action for same subject/rule/window; audit can show skipped duplicate if designed
Rule threshold	Admin/demo operator	Adjust threshold and rerun after reset	Result changes according to configuration; threshold remains labelled demo-only
Human decision boundary	Competent Authority / other role	Open Decision Pending case	Only competent-authority role gets decision controls; no automated decision

13. Team workstreams

Each workstream has one accountable owner, a clear output and a named integration partner. If the team has only three members, combine Workstreams B/C and assign integration ownership to the repository lead.

Workstream A — Product, workflow and integration lead

- Own the issue board, acceptance criteria, route/event/status names and merge decisions.
- Maintain docs and resolve conflicts before code is written.
- Review permission boundaries, deficiency return/resubmission and audit completeness.
- Own final end-to-end test and merge to main.**Done when:** P0 flow runs on main, all events/actions appear in audit, no duplicate automation actions, no known false claims.

Workstream B — Frontend / user experience

- Build navigation, overview metrics, application list/detail, filters and responsive UI.
- Use the shared data types/API contract; avoid inventing alternative field names.
- Surface state, owner, due date, next action, source rule and audit reference.
- Build clear blocked/empty/error states and visible simulation labels.**Done when:** every core user task has a working interaction and the UI reflects backend/source-of-truth state rather than local-only guesses.

Workstream C — Workflow, data and automation

- Implement route profiles, transition guards, required deficiency reason, resume stage, event handling and idempotency.
- Implement deadline/expiry evaluation, EVC queue logic, task/alert creation and rule version references.
- Design persistence/schema and audit transaction boundaries.**Done when:** positive, negative, boundary-date and duplicate-cycle tests pass.

Workstream D — Compliance, documents, EVC and QA (or combine with C)

- Implement evidence submission versus verification, renewal versus review, EVC scheduling/report tracking.
- Prepare synthetic cases and automated/manual scenario tests.
- Verify role controls and data flows end-to-end.**Done when:** all three lifecycle areas are traceable in UI and audit, and a teammate who did not code them can reproduce the demo.

Cross-team handshake

Before branches diverge, agree on: stage enum, route profile ID, role enum, status enums, due-date format, API response envelope, audit event shape, idempotency key inputs and ownership of each file. Every branch should have an issue, owner, acceptance criteria and integration dependency.

14. 24-hour build plan

Operating principle

Build and integrate one real vertical slice first. Aim for `event → rule → permitted state change/task → audit record`. Do not wait until hour 20 to integrate branches.

Time	Deliverable	Gate
H0–H1	Repo audit, dependency baseline, owners, branch plan, demo story	Every person knows files, owner and acceptance criterion

Time	Deliverable	Gate
H1–H3	Shared enums/types/contracts, synthetic data and basic app shell	Frontend and backend agree on exact shapes
H3–H6	Application list/detail, route-profile state model, first endpoint/state handler	A case can be read and a permitted transition is represented
H6–H9	Stage completion, blocked transition, deficiency return/resubmit	Reason required; resume stage correct; audit event appears
H9–H12	Rule engine + due date reminders/ escalations + idempotency	First cycle acts; second cycle does not duplicate
H12–H14	EVC queue, prerequisites, scheduling, report status	Eligible vs ineligible case behavior shown
H14–H16	Compliance evidence submit/verify and document renewal/expiry	Submit state cannot be mistaken for verification
H16–H18	Role checks, alerts/audit screens, filters	One coherent flow works through UI and data layer
H18–H20	Integration freeze, test all P0 and negative paths	No new P1 work unless all P0 passes
H20–H22	UX polish, responsive check, demo dataset reset, error handling	The demo is repeatable and readable
H22–H23	Two-person rehearsal, capture screenshots/ video	Speaker can explain each event and result
H23–H24	Regression, tag/backup, final limitations list	Known issues documented; no misleading claims

Scope cuts if late

Cut in this order: fancy charts/animations → nonessential filters → AI/OCR → live messaging → external integrations → extra route variations. Do not cut auditability, reason-required deficiency return, submit/verify separation, idempotency, core automation or role checks.

Checkpoint rules

- Main must remain runnable and demoable.
- Open a PR as soon as a small vertical slice is reviewable.
- Announce schema or enum changes before merging.
- At H18, freeze the contract. New features require the integration lead’s explicit approval.
- Record test commands and results; do not state that the app is production-ready.

15. Presentation source brief

This is factual source material for the teammate creating the PPT. It is not a slide deck. Confirm each claim against the current build before presenting it.

One-line proposition

ARGUS is an automation-first workflow and compliance intelligence platform that connects approval-stage routing, EVC scheduling, institutional undertakings, document validity, reminders/escalations and audit history in one role-aware console.

Problem framing

The SIH26262 statement describes coordination challenges in monitoring application movement through scrutiny/re-scrutiny, EVC, SAC/SHC and other applicable stages; tracking pendency; coordinating time-bound undertakings and compliance evidence; and monitoring lease/other document validity. It calls for configurable end-to-end workflows, role-based dashboards, timely alerts, escalations, reports and audit trails.

Do not assert unverified current failure rates or financial savings. The team may include an actual survey/interview only if it was genuinely conducted and the evidence can be shown.

Solution story

The system watches a defined workflow event, validates context and permission, evaluates configurable rules, creates a permitted next action, protects against duplicate side effects, and records an audit event. This is stronger than a static dashboard because the UI can show an explainable chain from trigger to outcome.

Key modules

1. Application routing and deficiency loop.
2. Pendency/aging and role-based work queues.
3. EVC eligibility, scheduling, report status and escalation.
4. Time-bound undertaking/compliance evidence with separate submit and verify states.
5. Document expiry and renewal tracking.
6. Configurable reminders/escalations and action queue.
7. Auditable actions, rules and reports.

Suggested slide sequence

1. Title: ARGUS — Complete Visibility. Accountable Governance.
2. Problem and workflow friction (use problem statement, not invented statistics).
3. Proposed solution: event → rule → authorized action → audit.
4. Workflow / role architecture.
5. Demonstrated automation (route advance + deficiency return + repeat-cycle idempotency).
6. Compliance and EVC / document expiry.
7. Security, human oversight and limitations.
8. Architecture and 24-hour build scope.
9. Demo evidence, next steps and production roadmap.

Demo evidence to capture

- Case stage before/after a permitted completion event.
- Deficiency return with required reason and resubmission path.
- First and second automation cycle showing no duplicate action.
- EVC queue and scheduling/report state.
- Evidence submission versus authorized verification.
- Expiry/renewal state and audit record.

Safe language

Say: “The prototype simulates rule-driven reminders and escalations with fictional records.” Do not say: “ARGUS is integrated with AICTE,” “the system verifies document authenticity,” “it sends real government alerts,” or “it reduces delay by X%” unless supported by implementation and independent evidence.

Final decision and statutory scrutiny remain with authorized officials; ARGUS supports coordination and traceability.

Appendix: Full-Stack Implementation Blueprint

Backend, Authentication, Shared Database, Automation Worker, Security and Deployment

Purpose: This is an implementation contract for AI coding agents and human developers. It converts the remaining gaps into a concrete build plan: backend/API server, real authentication, server-side authorization, shared persistent database, durable automation processing, audit trail, tests and deployment instructions.

Product identity: ARGUS — AICTE Approval & Compliance Intelligence Platform
Problem statement: SIH26262 — Integrated AICTE Approval Workflow Automation, Compliance Monitoring and Alert Management System
Tagline: Complete Visibility. Accountable Governance.

ARGUS is a proposed hackathon solution, not an official or endorsed AICTE product.

> **Implementation directive:** Do not stop at describing an architecture or making the UI look complete. Inspect the current repository, implement the agreed full-stack vertical slice, run the tests, and report precisely what works and what remains blocked by missing credentials or deployment access.

1. Current-state truth and target state

Current reference prototype

The current prototype is a self-contained HTML/CSS/JavaScript browser demo using synthetic records and browser-local state. Its role selection and simulated notifications are useful for demonstrating the workflow concept but are not production security or shared persistence.

Unless the repository audit proves otherwise, assume these capabilities are still missing and verify each one:

- Real login/logout and server-validated user identity.
- Server-side role and record-scope authorization.
- A shared database that persists changes across browsers/users.
- A backend API which is authoritative for workflow transitions.
- A scheduled worker with durable run records, idempotency and failure visibility.
- Real audit persistence and atomic relationship between mutations and audit events.
- Private evidence/document storage and controlled download access.
- Staging/production deployment configuration, observability, backups and operational instructions.

Never infer that a capability exists from a button or static demo. Prove it through code, tests and a repeatable run.

Target for the next implementation

Build a **full-stack, multi-user hackathon MVP** with real authentication, shared PostgreSQL persistence, server-validated permissions, automation rules that can be rerun safely, an auditable workflow and a working frontend connected to the API. This is a production-shaped prototype, not a claim of production certification or permission to process real regulatory records.

2. Repository-first rule and default stack

Mandatory first step

The coding agent must inspect the existing repository before changing its framework. Identify the current frontend, backend, dependency manager, scripts, migrations, tests, hosting setup and environment variables. Preserve a working stack if one already exists. Do not delete or replace the current UI just to modernize it.

Recommended default only if no functioning backend/auth/database stack exists

- **Frontend:** retain the existing HTML/CSS/JavaScript prototype initially and split it into maintainable files only where useful. Use the existing framework if the repository already has one.
- **Backend API:** Python FastAPI, with typed request/response schemas and a service layer for workflow rules.
- **Identity provider:** Supabase Auth or another explicitly approved identity provider. Use real provider-managed sign-in/session handling, not a client-side role dropdown.
- **Shared database:** PostgreSQL, preferably the project's Supabase PostgreSQL instance for quick shared development. Use versioned migrations; SQLAlchemy/Alembic is the suggested Python path if suitable for the existing repository.
- **Evidence storage:** private object storage with scoped access; for a Supabase-based build, use a private Storage bucket and authorization-aware access policies.
- **Automation worker:** a separate worker process or managed scheduled job that evaluates due dates/rules. A development-only manual trigger may exist for testing, but it is not a substitute for the scheduled worker.
- **Notifications:** persisted in-app notifications first. External email/SMS delivery is an optional adapter and must be labelled simulated until provider credentials and delivery tests exist.

This is a pragmatic default, not an instruction to introduce every service regardless of need. If the current repo already has a working, secure equivalent, preserve it and record the decision in `docs/DECISIONS_AND_ASSUMPTIONS.md`.

3. Required architecture

The required logical separation is:

1. **Web client:** renders case lists, details, action forms, queues, deadlines, notifications and audit history. It never decides whether a user is authorized or whether a workflow transition is valid.
2. **Authentication:** provides login, logout, session refresh and verified identity claims. The client sends an access token to the API over HTTPS.
3. **API server:** verifies every token and validates each command. It is the authority for domain mutations and returns stable error codes.
4. **Authorization/policy layer:** resolves the user's active role, institution/region/case scope, allowed actions, route preconditions and relevant policy version.
5. **Domain services:** application routing, deficiency return/resubmission, EVC scheduling, compliance evidence verification, document renewal and audit/query services.
6. **PostgreSQL database:** stores users/profiles, scoped memberships, cases, route state, obligations, evidence metadata, rules, alerts, idempotency records, worker runs and audit events.
7. **Automation worker:** performs due/expiry scans, evaluates active rule versions, creates permitted tasks/alerts, tracks failures and retries safely.
8. **Notification outbox/adapter:** stores the intention to notify in the same transaction as the relevant state change, then delivers in-app or external notifications asynchronously.
9. **Private file storage:** keeps actual uploaded documents separate from relational metadata; authorization is checked before upload, download and review.
10. **Operations:** structured logs, request/correlation IDs, health checks, migration command, backup/restore instructions and visible failed jobs.

Workflow command path

```
HTTP request -> token verification -> role/scope authorization -> input validation -> load current row/version -> validate state transition and prerequisites -> evaluate configured rule/policy version -> transactionally persist state + audit + idempotency/outbox record -> worker/notification processing -> updated UI and traceable audit
```

All important checks happen on the server, even if the frontend disables an action button. Do not trust browser-local state, a role field sent by the client, or a decoded-but-unverified token.

4. Authentication and authorization requirements

Authentication

- Implement login, logout, session restoration/refresh and a clear unauthenticated state.
- Verify the provider's access token on the server using the provider's documented verification method for the configured signing-key format. Validate signature, issuer, audience and expiry. Never merely base64-decode a JWT and treat its contents as true.
- Use invite/admin provisioning for privileged demo roles. Users cannot grant themselves an officer, reviewer, authority or administrator role through a signup form or local browser state.
- Provide synthetic development/test accounts through documented seed/admin setup. Do not place real passwords or production credentials in Git.
- Protect secrets with environment variables/secret management; keep server-only credentials out of frontend bundles and public files.

Authorization

Every API handler and every sensitive object operation must enforce both **action permissions** and **record scope**. A valid login alone is not sufficient.

Role	Allowed capabilities (subject to assigned scope)	Explicit restriction
Institution user	View own institution's cases/obligations; respond to deficiencies; submit evidence and renewal items	Cannot verify its own evidence, change policy or perform officer-only transitions
Scrutiny officer	Review assigned eligible cases; record observations; perform allowed scrutiny transitions; return for deficiency with reason	Cannot perform actions outside assigned scope or make decisions reserved to the competent authority
EVC coordinator/member	View eligible EVC queue; assign/schedule visits; track visit/report state	Cannot bypass route prerequisites or verify unrelated obligations
Compliance reviewer	Review submitted evidence and renewal items; record verification outcome and notes	Cannot verify evidence submitted by the same institutional identity where separation of duties applies
Regional/Bureau officer	Monitor permitted region/workload; assign follow-up; escalate according to policy	Cannot access other regions or edit global policy unless separately authorized
Competent authority	View complete authorized case file and record the reserved decision	The final decision must be an explicit human action with an audit trail, never an AI-generated automatic decision
Policy administrator	Manage route profiles/rule versions within granted scope; enable/disable configuration	Policy changes require validation, versioning and audit; admin role must not imply unlimited access to evidence content

The implementation may adapt role names to the existing canonical schema but must preserve the capability boundaries. Document any role/schema changes and update tests.

Tenant/record isolation

- Every institution-owned record must be scoped to an institution ID; region-bound roles must be scoped to authorized regions.
- Prevent insecure direct object reference (IDOR) bugs: changing an ID in a URL must not expose or mutate another tenant's records.
- Add database row-level security (RLS) or an equivalently verified database-level isolation strategy in addition to API authorization. Document exactly which paths use RLS and which checks are enforced in the server.

- Do not expose a service-role/admin database credential to the browser. If a server-only privileged credential bypasses RLS, the API must enforce authorization for every read/write and tests must prove isolation; prefer least-privileged DB roles and RLS wherever feasible.
- Return `403 Forbidden` for an authenticated user without permission, and avoid leaking sensitive existence/details when the caller has no access.

5. Shared database and schema requirements

Use PostgreSQL migrations under version control. Do not make the schema depend on manually clicking together tables in a dashboard. A new teammate must be able to create the schema from documented commands.

Minimum logical tables/entities

- `user_profiles`: auth-provider user ID, display metadata, account status, timestamps.
- `role_assignments` or `memberships`: role, institution/region scope, start/end dates, assigned-by and audit metadata.
- `institutions`: synthetic institution identifier and non-sensitive demo metadata.
- `applications`: institution, application type, route profile/version, current stage/status, owner/scope, due date, row version and timestamps.
- `route_profiles`, `route_stages` and `transition_rules`: configured paths and prerequisites; no universal hard-coded route.
- `stage_history`: from/to stage, actor/system identity, reason, event time, policy version and correlation ID.
- `deficiency_requests`: reason, missing items, stage to resume, status and response linkage.
- `evc_assignments` / `evc_events`: eligibility, coordinator, visit time, committee/panel metadata and report status.
- `compliance_obligations`: obligation type, source/policy reference, institution, due date, owner and state.
- `evidence_submissions`: private storage key, submitter, submitted-at time, review state, verifier, decision and notes.
- `documents` / `renewal_submissions`: document type, validity dates, storage key, submission/review status and review history.
- `automation_rules` and `automation_rule_versions`: enabled state, trigger, configured threshold, escalation route, source/effective dates, creator and approval metadata.
- `automation_runs` / `automation_run_items`: worker start/end/status, evaluated subjects, outcomes and failures.
- `idempotency_records`: unique key for command/event processing and result/reference.
- `alerts` / `notifications`: target user/role, trigger/rule version, severity, status, created/acknowledged/resolved times and delivery channel status.
- `notification_outbox`: durable send intent, destination reference, attempt count, next retry, status and last error; never store secrets in payloads.
- `audit_events`: append-only logical history with actor/system, action, entity, timestamp, outcome, reason, rule version, correlation ID and minimal safe metadata.

Do not duplicate unnecessary PII into the event log. Use foreign keys, indexes for common filters and unique constraints for idempotency. Use timestamps in UTC and present them in the user's timezone in the UI.

6. API and transaction contract

Use a versioned API namespace such as `/api/v1`. Exact framework syntax may vary, but behavior must be equivalent to this contract.

Required read endpoints

- `GET /health` — no sensitive configuration in the response.
- `GET /api/v1/me` — verified profile and permitted role/scope summary.
- `GET /api/v1/dashboard` — counts/aging derived only from the caller's authorized scope.
- `GET /api/v1/applications` and `GET /api/v1/applications/{id}`.

- `GET /api/v1/evc-queue` and `GET /api/v1/evc/{id}`.
- `GET /api/v1/compliance-obligations` and `GET /api/v1/evidence-submissions`.
- `GET /api/v1/documents` with expiry/renewal filters.
- `GET /api/v1/alerts` and `GET /api/v1/audit-events` with authorization and pagination.
- `GET /api/v1/rules` for allowed rule metadata; only policy administrators may read/edit restricted settings.

Required mutation endpoints

- Create intake/application; server chooses the configured initial state.
- Complete a stage; server validates role, current row version, route prerequisites and allowed next stage.
- Return for deficiency; requires a non-empty reason and persists the stage to resume.
- Resubmit response/evidence; returns to the designated review point, not an arbitrary client-selected stage.
- Schedule EVC and submit/record report separately.
- Submit evidence; resulting state is `Pending Verification`.
- Verify/reject/request clarification; only an authorized, non-conflicted reviewer may decide.
- Submit renewal and verify renewal as separate operations.
- Run an automation cycle through a test/admin-only trigger in development; production schedule runs through the worker identity.
- Acknowledge an alert without implying the underlying obligation is resolved.
- Create a new rule version through a privileged, validated and audited operation.

API rules

- Use typed schema validation and stable error codes: `VALIDATION_ERROR`, `UNAUTHENTICATED`, `FORBIDDEN`, `NOT_FOUND`, `CONFLICT_VERSION`, `INVALID_TRANSITION`, `PREREQUISITE_NOT_MET`, `DUPLICATE_ACTION`, `POLICY_INVALID`, `RATE_LIMITED`, `TEMPORARY_UNAVAILABLE`.
- Use optimistic concurrency (`row_version`/expected version) for case mutations.
- State-changing operations must not be implemented as `GET` requests.
- Mutations must persist domain state and audit information in one database transaction. Use an outbox for asynchronous messages.
- Return a request/correlation ID on success and error. Do not return raw stack traces or secrets to the client.
- Apply pagination and server-side filters; do not download every tenant's case list to the browser and filter locally.

7. Automation worker, reminders and notifications

Worker behavior

- Implement a separate callable worker/cron job that evaluates due dates and active rule versions on an explicit schedule.
- Persist each run and per-subject result. A failed item must be visible, logged with a correlation ID and retryable where safe.
- Generate an idempotency key with the subject, rule ID, threshold window and rule/policy version. Enforce uniqueness in the database; an in-memory set is not enough.
- Acquire work safely under concurrent workers (row locks/atomic claims or an equivalent design). Two worker processes must not create duplicate actions.
- Store the alert/task/audit/outbox intent before attempting external delivery. Retry transient failures with bounded backoff; report exhausted retries in a failed-job surface.
- Keep the development-only manual run endpoint behind authentication and an admin/demo permission. Production schedule must not depend on a human clicking a button.
- Avoid hard-coded legal timeline assumptions. Configured demo thresholds must say they are illustrative until validated against currently applicable official rules and authorized procedures.

Notifications

- P0: real database-backed in-app alerts that appear for the correct authorized user/role and persist after reload/login from another browser.
- Optional: email/SMS adapter only after provider configuration, secret handling, error/retry tracking and a real delivery test. Until then, label external delivery as simulated.
- Record delivery outcomes separately from the underlying alert. An email failure must not roll back an already persisted case transition.

8. Evidence and document storage

- Store metadata in PostgreSQL and file bytes in a private object store, not as public URLs.
- Validate permitted file types, size limits and metadata; do not trust filename extensions or client MIME declarations alone.
- Generate short-lived signed access links or stream files through an authorized endpoint after checking identity and scope.
- Log upload, download, review and export activity without logging the full document contents.
- Use synthetic, non-sensitive evidence in the hackathon. Malware scanning, retention controls and formal document authenticity checks are separate tasks; do not claim they exist unless implemented and tested.

9. Production-shaped security baseline

Before the app can be described as ready for real data, address at minimum:

- HTTPS at every deployed boundary and exact allowed CORS origins.
- Environment-based configuration and a `.env.example`; real secrets only in secret storage.
- No hard-coded credentials, service keys or personal data in Git.
- Authentication token validation, session expiry and logout behavior.
- Server-side action/record-scope authorization and tenant isolation tests for every role.
- Request validation, output encoding, rate limiting and safe error messages.
- CSRF/session protections appropriate to the chosen auth mode.
- Safe file upload/download and private storage.
- Database backups and a demonstrated restore procedure before production use.
- Dependency scanning/updates, structured logs, failed worker visibility and health checks.
- Admin/policy change audit, access review, incident-response owner and defined retention policy.

This baseline is engineering guidance, not security certification or a legal compliance opinion.

10. Deployment and configuration

Provide a `docker-compose.yml` or equivalent reproducible local environment if practical, plus separate instructions for local development and hosted staging. Keep hosting details adaptable to team access; do not hard-code a provider into the application logic.

Minimum config documentation:

- `SUPABASE_URL` / auth issuer URL.
- Public/anonymous auth key for browser usage only if the chosen provider requires it.
- Server-only privileged credential, if required by the selected architecture, clearly labelled secret and never included in frontend bundles.
- `DATABASE_URL` or documented provider DB connection configuration.
- JWT/JWKS verification configuration as required by the auth provider.
- `APP_ENV`, `APP_BASE_URL`, `ALLOWED_ORIGINS` and `LOG_LEVEL`.

- Private document-storage configuration.
- Notification provider settings only if real delivery is implemented.

The `.env.example` must contain placeholders, never usable secrets. Document database migration, seed, API start, worker start, tests, frontend start, and staging deployment commands. Configure health checks, logs and alerts for API/worker failures. Treat migration rollback/backup risks explicitly.

11. Test and acceptance plan

No feature is complete because a route returns HTTP 200. Add tests around actual persistence, permissions and workflow outcomes.

Authentication and authorization

- Anonymous user cannot read protected data or mutate a case.
- Expired, malformed and invalidly signed tokens are rejected.
- User cannot choose/change their role via request payload or browser state.
- Institution A cannot view or modify Institution B's case, evidence, alerts or document.
- Region-limited officer cannot access a different region.
- Institution user cannot verify own evidence; only authorized reviewers can verify.
- Competent-authority decision endpoint rejects other roles.
- Admin route/rule changes require privileged permission and generate an audit event.

Workflow and persistence

- Valid route transition succeeds and persists after restart and in a second browser/session.
- Invalid transition and unmet prerequisite are blocked by the backend.
- Deficiency return requires a reason and resubmission returns to the correct review stage.
- EVC eligibility, scheduling, visit and report status are distinct.
- Evidence submit creates pending verification; verification requires an authorized actor and note/outcome.
- Document expiry produces one warning per configured window; renewal submission does not equal verification.
- Every mutation creates a correlated audit event transactionally.
- Concurrent requests cannot silently overwrite newer case state.

Automation and delivery

- Running the same automation cycle twice creates no duplicate alerts/actions.
- Two concurrent worker instances do not double-process the same idempotency key.
- Transient delivery failure is retried; exhausted retries are visible.
- Alert creation persists even if an external notification provider is unavailable.
- Rule version changes affect only the intended policy version/effective window.

Release gate

The team must demonstrate the flows with synthetic users of different roles, use two separate browser sessions, restart the services and prove that shared state remains consistent. Record the exact commands and results in `docs/VALIDATION_RECORD.md`. If credentials/hosting are unavailable, label the related gate `BLOCKED BY CONFIGURATION` rather than pretending it passed.

12. Recommended implementation order for the 24-hour sprint

This order prioritizes a connected vertical slice. Re-estimate after the repository audit; do not skip tests to maintain an unrealistic schedule.

1. **Audit and freeze contracts (Hour 0-1):** inspect stack/scripts, choose default stack only if necessary, record decisions, set env variable names, agree on roles, state names, APIs and ownership.
2. **Shared DB (Hours 1-4):** define migrations, foreign keys, indexes, role/scope tables, core workflow tables, audit/idempotency/outbox tables and synthetic seed data. Verify a clean reset/migration.
3. **Authentication (Hours 3-6):** provider login/logout/session, verified API identity, protected route and role loading; remove or prominently isolate fake role-switching from the authenticated experience.
4. **Server-authoritative workflow (Hours 5-10):** route-aware advance, invalid transition blocking, deficiency return/resubmit/resume, version conflict and transactional audit.
5. **EVC/compliance/document APIs (Hours 8-13):** scheduling/report status, evidence submit/verify separation and renewal/expiry lifecycle.
6. **Automation and in-app alerts (Hours 11-16):** scheduled worker/manual dev trigger, idempotency uniqueness, alerts, outbox and failure visibility.
7. **Connect frontend (Hours 14-19):** loading/error/empty states, real API calls, role-aware actions, cross-session persistence and honest integration status labels.
8. **Security/test pass (Hours 19-22):** permission matrix, tenant isolation, duplicate/concurrent run, boundary dates, migrations and build regression.
9. **Integration/rehearsal (Hours 22-24):** fresh setup, two-user scenario, final demo script, backup recording/screens, known-limits list and release notes.

If the shared backend/auth stack cannot be completed safely in the window, keep the static simulation clearly labelled and ship the backend work as an explicit branch or incomplete phase. Never present a non-integrated mock as real authentication or shared persistence.

13. Definition of done for coding agents

The implementation task is complete only when the agent provides:

1. Repository audit and stack decision (existing stack preserved or reasoned change).
2. Actual code changes for assigned scope, not documentation alone.
3. Database migrations and synthetic seed/reset instructions.
4. Authentication/token verification and server-side role/scope enforcement for protected operations.
5. The relevant API, UI and database wired together end-to-end.
6. Tests for positive, negative, unauthorized, cross-tenant, retry/idempotency and boundary-date paths where relevant.
7. Exact commands executed and real test/build outputs; no invented test success.
8. A concise list of implemented, unimplemented and credential-blocked capabilities.
9. Updated README, `.env.example`, API/architecture docs and validation record.
10. A runnable clean-setup path another teammate can follow without verbal instructions.

Required task-response format

- Objective and source requirements.
- Current repository finding.
- Files inspected/changed.
- Design and API/database contracts.
- Acceptance criteria.
- Commands run and exact results.
- Known gaps, configuration blockers and next integration owner.

14. Copy/paste execution prompt for a coding agent

> You are implementing ARGUS, the SIH26262 approval workflow and compliance automation platform. Read `AGENTS.md`, `README.md`, `docs/PROJECT_CHARTER.md`, `docs/SOLUTION_SPECIFICATION.md`, `docs/24_HOUR_MVP.md`, this `docs/PRODUCTION_IMPLEMENTATION_BLUEPRINT.md`, and all relevant files under `technical/` before editing. First inspect the existing repo, scripts, dependencies, tests, environment variables and current prototype. Report which components actually exist and which are mock-only. Preserve a functioning stack unless the audit shows there is none; if no backend/auth/shared DB exists, use the recommended stack in this blueprint and record the decision. Then implement the first end-to-end slice: real provider-backed login, verified API identity, server-side role and record-scope authorization, a shared PostgreSQL schema via migrations, one server-authoritative application transition, a persisted audit event, a real database-backed in-app alert, and idempotent automation. Build in small commits; don't merely create more plans. Add migration/seed/setup commands, update `.env.example`, connect the frontend, and test anonymous/forbidden/cross-tenant/invalid-transition/duplicate-cycle cases. Continue to EVC, compliance evidence, document renewal, worker scheduling and deployment hardening in the order above. Do not expose server credentials, fake real email/SMS, invent legal timelines, or allow AI to make statutory decisions. At the end, report exact changed files and commands/results, plus what is blocked by missing credentials. Never call a simulated component production-ready.